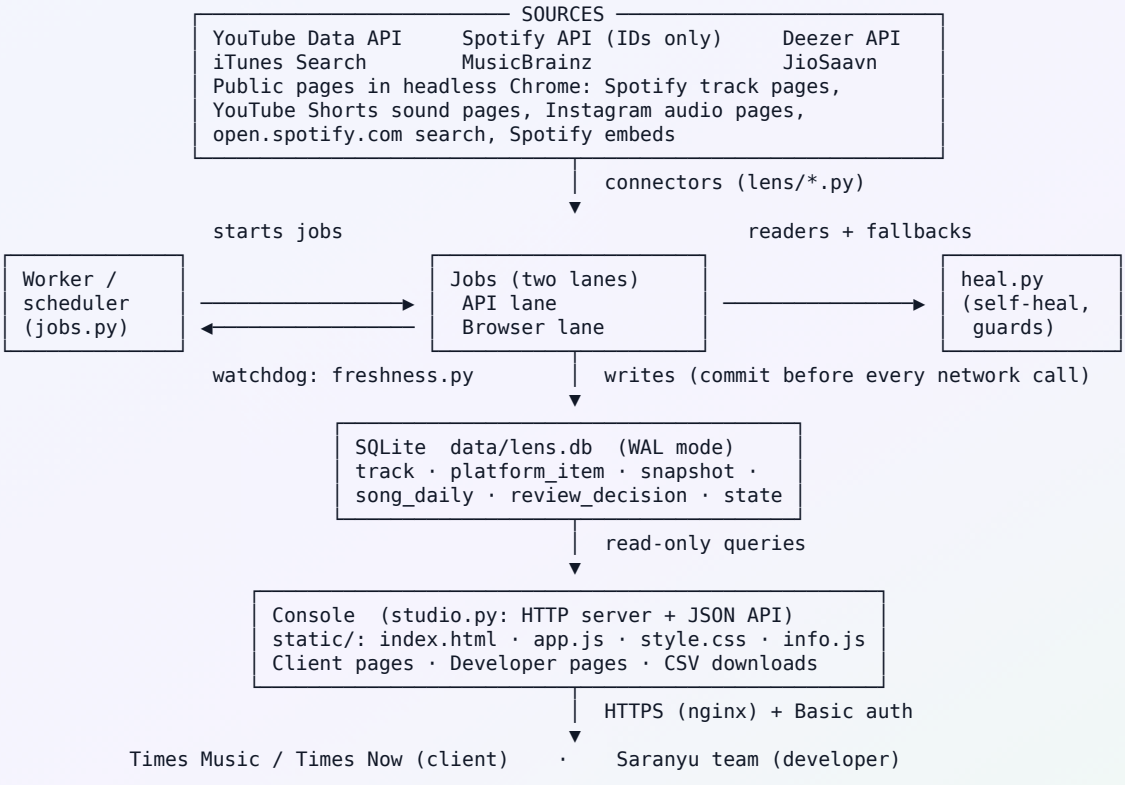
Market Lens: architecture and how it works

Saranyu Technologies, for Times Music. Updated 2026-11-12.

Market Lens tracks new Indian film and music releases across platforms. It finds every new song on the record labels' YouTube channels. It then finds the same song on Spotify, YouTube Music, JioSaavn, Deezer, Apple Music, MusicBrainz and Instagram, and reads each song's numbers on every platform once a day. The results feed a web Console for the client and developers, CSV downloads, and a model for Power BI.

- **Scope:** 7 labels and 84 YouTube channels: T-Series, Saregama, Zee Music Company, Sony Music India, Times Music, Aditya Music and Think Music.
- **Which songs:** songs first uploaded on YouTube from 2026-07-01 onwards. A song released before that date costs no crawl calls; it is kept for reference only.
- **Ground rules:**
  - No AI or ML: all matching is rule-based.
  - Python 3 standard library only, with one SQLite database.
  - Daily data is never deleted.

1. The big picture



2. The data flow, step by step

Step 1: Discover songs on YouTube (`sync_youtube.py`, `youtube.py`)

- Reads the new uploads of every label channel, about 30 quota units a sync.

- Reads views, likes and comments: every 8 hours in a video's first 7 days, then once a day. A read costs 1 unit per 50 videos.
- Uses 8 API keys, one per Google Cloud project (about 80,000 units a day). A key whose quota runs out is set aside until midnight Pacific time.

Step 2: Group uploads into songs (`songs.py`, `merge.py`)

- **One song, many uploads:** one song is many YouTube uploads (video, lyric, audio, Shorts, teaser). Each upload is a `platform_item`; the song is a `track`.
- **Labels stay apart:** songs from different labels never merge.
- **Merge rules:** configurable rules merge duplicates. Every merge is logged and can be undone.

Step 3: Find the song on the other platforms (matching)

Platform	How the ID is found	How the numbers are read
Spotify	1) Label sweep (`spotify_labels.py`): album search per label, then every track scored. 2) Per-song API search on 2 accounts. 3) Logged-out browser search on open.spotify.com.	Public track page in headless Chrome (`spotify_plays.py`). **No API calls after the ID.**
YouTube Music	One search per song, plus artists' "- Topic" channels (`youtube_music.py`)	Plays
JioSaavn	Search (`jiosaavn.py`)	Play count
Deezer	By ISRC (`api.deezer.com/track/isrc:…`)	Rank, daily
Apple Music / MusicBrainz	iTunes Search / MusicBrainz	Metadata: release date, genre
YouTube Shorts	The song's official sound page (`shorts_use.py`)	Number of Shorts using the sound
Instagram	Audio link pasted by a developer (`instagram_audio.py`)	Number of Reels using the audio

Match scoring (`match.py`) uses these weights:

Part	Weight
Title	0.55
Artist	0.15
Film	0.10
Duration	0.10
Date	0.10

- **Automatic link:** a total of at least 0.75 with a title score of at least 0.85.
- **Review:** a total of at least 0.55 sends the match to Review.
- **Never matched:** remix, male/female and "X version" variants.
- **Extra guard:** a track links automatically only if its own title shares a word with the song.

Step 4: Human review (`review.py`)

Uncertain matches wait in the Console's Review page. Every approve, reject or merge is stored in `review_decision`, and each one can be undone exactly.

Step 5: Daily readings and roll-up (`song_daily.py`)

- **Raw readings:** each reading is stored as a snapshot.
- **Daily roll-up:** `song_daily` holds one row per song per day, with YouTube views, Spotify plays, YouTube Music plays, JioSaavn plays, Deezer rank, Shorts and Reels.
- **Longer periods:** the weekly, monthly and yearly views build on `song_daily`.

Step 6: Metadata (`metadata.py`, `release_year.py`)

- **Credits:** parsed from the YouTube descriptions.
- **Other details:** Spotify, Apple and MusicBrainz data, plus the language, category and occasion rules. They are stored in `song_meta`.
- **Release date:** the release-year check marks songs that were released before the period, so they stop costing crawl calls.

Step 7: Present (`studio.py` + `static/`)

The Console serves JSON APIs and a single-page UI, described in section 5.

3. Jobs, lanes and scheduling (`jobs.py`)

A **worker** loop picks the next job that is due. It runs **two lanes at once**, so slow browser reading never blocks API work.

Lane	Jobs
API lane	`pipeline` (YouTube sync, songs, YouTube Music, JioSaavn, daily roll-up), `youtube_sync`, `songs`, `spotify_label_sweep`, `spotify_primary_rediscover`, `spotify_second_account`, `ytmusic`, `jiosaavn`, `shorts`, `cleanup`
Browser lane (headless Chrome)	`spotify_plays`, `spotify_browser_search`, `instagram`

Readings within 24 hours (`freshness.py`). This watchdog is the first check in every worker loop.

- **Missed items:** if any song on any linked platform has not been read in 24 hours, that platform's job starts at once, retried every 30 minutes.
- **Stuck platforms:** after 3 runs that read none of the missed items, the job is retried every 3 hours and the Console marks it "stuck".
- **Console page:** the missed items are listed on Missed readings.

Pause and resume (`pause.py`).

- **Pause:** running jobs finish and nothing new starts.
- **Pause now:** running jobs also stop at their next safe point, between songs or pages, keeping the data read so far.
- **Safety:** the pause is stored in the database, so it survives a restart, and it always ends by itself.
- **Deploys:** the deploy script uses Pause now, so a deploy never interrupts a crawl.

Quota protection.

- **Spotify accounts:** each account has a counted daily limit in `data/spotify_calls.db`: 700 per account on the server, at most one call a second.
- **One search a day:** a song is searched at most once a day across both accounts.
- **On a 429:** an account that gets a 429 waits until Spotify's Retry-After time.

Daily cleanup (`cleanup.py`, after 03:00 server time).

- **Backup:** gzipped, 14 days kept.
- **Raw replies:** those older than 7 days are moved to `data/archive/`.
- **Disk space:** free pages are given back to the disk.
- **Nothing is lost.**

4. Reliability: self-healing and failure-proofing

Risk	Protection
A platform changes its page	`heal.py`: every value has 2–4 fallback reading strategies. The strategy that worked is counted per day, and a marker that works is learned. One failing page a day is saved for debugging.
A broken reader marks songs as "read" with no number	Guards: a reader that stops working never marks items read. "Hidden" counts are trusted only if songs that usually show a count still do. Otherwise the batch is held and read again.
A wrong number gets stored	Suspicious values (a fall of more than 10%, or a jump of more than 30×) go to `reading_suspect`, not into the data.
The database is locked between jobs	Every loop that calls the network commits before each call. A job's result is saved with retries, and its lane is always freed.
Timeouts	Deezer retries 3 times and stops only after 5 failures in a row. Other readers retry the next run.
A Console page fails	The page shows "This page could not load... Retry" instead of a blank screen.
Something goes wrong silently	Alerts (`alerts.py`): `heal:<reader>` (warning or critical) and `suspect:<reader>`, on the Developer dashboard.

There are 31 automated tests (`python3 -m unittest discover lens/tests`, run from `lens/`). They run on real saved pages and on deliberately changed copies of them.

5. The Console (web UI)

Logins (HTTP Basic auth, enforced on the server):

Role	Who	Can see
Client (shared)	Any username with `STUDIO_PASSWORD`	Dashboard, Management, Market share, Platforms, Reports, Daily data, Songs (read-only)

Role	Who	Can see
Named client	Own username and password, e.g. `timesnow` (`logins.py`; only a PBKDF2 hash is stored)	Same as the shared client login; each account can be removed on its own
Developer	`DEVELOPER_USERNAME` / `DEVELOPER_PASSWORD`	Everything: Review, Runs, Job control, Missed readings, Daily additions, Sources, Settings, Outputs

The server checks every API: a client calling a developer API gets 403, and every POST request is developer-only. Instagram data is stripped from every client reply and CSV.

Hover help: hovering any title, number, column or button explains what it is, how it is worked out, when it updates and why it matters (developers also see the technical detail).

Main pages:

- **Insights:**
  - **Dashboard:** YouTube views, audio streams, ratios, top songs and fastest growing songs.
  - **Management:** a weekly summary ranked by Spotify plays gained.
  - **Market share:** each label's share of the growth over a song's first N days.
  - **Reports:** labels day by day (with a heatmap) and month by month.
- **Data:**
  - **Platforms** and **Songs.**
  - **Daily data:** per song, per platform.
  - **Outputs:** datasets and CSV downloads. **CSV downloads never contain IDs or links.**
- **Operations (developer):**
  - **Developer dashboard:** Phase 0 targets, quotas, job control and storage.
  - **Runs** and **Missed readings** (with the Page readers card).
  - **Daily additions:** songs found per platform per day, with One platform, Compare, Head to head and vs YouTube views.
  - **Review.**
- **Setup (developer):** Sources, Collection schedule and Settings.

Charts: line, bar, donut, ranked bars, stacked bar, stacked columns and heatmap. Many charts have a switch between forms, and the choice is remembered per browser.

Look: the colours of the Market Lens logo, with a slow aurora behind the pages and semi-transparent frosted cards, in light and dark themes.

6. Code map (`lens/lens/`)

Area	Modules
Core	`config.py` (config.json, .env, period window), `db.py` (schema, connections)
YouTube	`youtube.py`, `sync_youtube.py`, `youtube_archive.py`, `shorts_use.py`
Songs and matching	`songs.py`, `match.py`, `merge.py`, `review.py`, `release_year.py`, `metadata.py`

Area	Modules
Spotify	`spotify.py` (API client and call counter), `spotify_labels.py`, `spotify_artists.py`, `spotify_browser_search.py`, `spotify_found.py`, `spotify_links.py`, `spotify_plays.py`, `spotify_playlists.py`, `spotify_list.py`, `spotify_charts.py`, `discover_spotify.py`
Other platforms	`youtube_music.py`, `jiosaavn.py`, `instagram_audio.py` (Deezer, iTunes and MusicBrainz inside `songs.py`)
Operations	`jobs.py`, `pause.py`, `freshness.py`, `heal.py`, `alerts.py`, `health.py`, `cleanup.py`, `sysstats.py`, `crawl_stats.py`, `live_status.py`
Reports	`song_daily.py`, `management.py`, `market_share.py`, `label_report.py`, `additions.py`, `insights.py`, `exports.py`, `collection.py`
Console	`studio.py` (server and API), `logins.py`, `static/` (index.html, app.js, style.css, aurora.js, info.js: hover help)
Tests	`tests/test_heal.py` (27), `tests/test_logins.py` (4)

Main tables:

- **Songs and items:**
  - `track`: a song.
  - `platform_item`: an upload or track on a platform, with its review state.
- **Readings:**
  - `snapshot` / `reading`: raw readings.
  - `song_daily`: the daily roll-up.
  - `sound_use` / `sound_short`: Shorts and Reels.
- **Review and merges:** `review_decision`, `song_alias`, `merge_rejection`, `match_rejection`.
- **Spotify discovery:** `spotify_label_album`, `spotify_label_track` and `spotify_found`.
- **Operations:** `run_log`, `state`, `alert`, `pause_log`, `read_mark`, `extract_stat`, `reading_suspect`.
- **Archive:** `raw_capture` (raw API replies) and `song_archive` (songs outside the period).

7. Deployment

- **Server:** an AWS ARM instance. The app is in `/home/ubuntu/lens` and served at `https://demomusic.apisaranyu.in`; nginx serves HTTPS and redirects HTTP to it. The Console listens on port 3000 behind it.
- **Start command:** `python3 -m lens.studio --host 0.0.0.0 --port 3000`. The worker runs inside the same process.
- **Deploy:** run `./deploy_server.sh`. It does five things in order:
  - Pause now.
  - Wait until both lanes are idle.
  - Back up the database.
  - Copy the code and restart the Console.
  - Resume the jobs, retrying for up to a minute.

- **Backups:**
  - **Nightly:** a cron job at 02:30, with the cleanup job as a fallback.
  - **Before each deploy:** an extra copy.
  - **Before an `.env` edit:** `.env.bak.<timestamp>`.
- **Secrets:** `.env` on the server (chmod 600) holds the API keys, the passwords and the named-login hashes.

8. Phase 0 targets (shown on the Developer dashboard)

Target	Goal
Releases discovered within 24 h	≥ 95%
Matched across platforms	≥ 90%
Daily readings captured	≥ 99%
Matches needing manual review	≤ 10%

© 2026 Saranyu Technologies. All rights reserved.